Universidad Tecnológica Nacional
Tecnicatura Universitaria en Programación

Materia: Proyecto Final

EasyPool

Sistema web para la gestión de agenda, asignación de técnicos y seguimiento de servicios de mantenimiento de piscinas

Informe Final del Proyecto
Grupo 163 · Año 2026

Integrantes

Tomás José Buforn
Maximiliano Leonel Rao
Eric Suarez Dubs

Tutor: Oscar Londero

Versión evaluada: v1.0-entrega-final - Repositorio: https://github.com/MaximilianoRao/EasyPool

Video de presentación: https://youtu.be/ZVagtkr7ObY

Resumen

EasyPool es una aplicación web que centraliza la agenda de una pyme de mantenimiento de piscinas: el administrador carga clientes, ubicaciones y planes de mantenimiento, el sistema genera los servicios recurrentes, el administrador asigna técnicos y cada técnico consulta y actualiza el estado de sus servicios desde el celular. Los cambios de agenda llegan al técnico como avisos dentro de la aplicación, sin pasar por mensajes manuales.

El sistema está implementado (frontend React, backend Node.js/Express, base PostgreSQL), desplegado en línea, cubierto por 78 pruebas automatizadas y evaluado en un piloto simulado que compara el proceso manual actual (Excel y WhatsApp) con el uso del sistema. El piloto muestra que el flujo completo funciona de punta a punta y que la coordinación fuera de la aplicación se elimina en el escenario simulado; el ahorro de tiempo administrativo medido (23 % a 26 % en total) no alcanza la meta de 50 %, y la validación con una empresa real queda pendiente.

1. Problema y contexto
2. Objetivos y alcance
3. Análisis y modelo
4. Arquitectura y tecnologías
5. Implementación
6. Pruebas y validación
7. Resultados y conclusiones
8. Enlaces de la entrega

1. Problema y contexto

Situación que dio origen al proyecto

El caso está inspirado en un negocio de mantenimiento de piscinas observado informalmente por el equipo. Su operación diaria se apoya en planillas de Excel para planificar los turnos y en WhatsApp para comunicarse con el personal de campo. Cuando hay un cambio de último momento (una cancelación, una reprogramación, un técnico que no puede ir), el administrador debe actualizar la planilla y avisar uno por uno a los técnicos afectados. La información queda repartida entre varios medios y no hay una vista única y actualizada del estado de cada servicio.

Actores involucrados

Actor	Necesidad
Dueño / administrador	Planificar con menos esfuerzo, reducir errores de asignación y ver la operación del día en un solo lugar.
Personal de campo (técnicos)	Ver su agenda actualizada, enterarse de los cambios sin depender de mensajes manuales y reportar el avance de cada servicio desde el teléfono.
Clientes	Previsibilidad sobre la visita. No son usuarios del sistema en esta versión: el administrador sigue comunicándose con ellos por los medios actuales.

Evidencia utilizada para validar el problema

La evidencia es limitada y se declara como tal: se trata de una **observación informal** de un negocio real del rubro. No hubo entrevista formal con el dueño, ni encuesta, ni medición del proceso real. Las cifras de partida (unas 4 horas diarias de planificación y entre 2 y 3 horas de retrabajo por cambios) son una **estimación del**

equipo, tratada como hipótesis de trabajo y no como dato confirmado. Por ese motivo el proyecto no mide contra una línea de base real: compara dos procesos simulados con el mismo escenario (manual contra sistema), lo que da una estimación relativa y no reemplaza una medición con la empresa.

Necesidad concreta

Reducir el esfuerzo administrativo de planificar y coordinar servicios, y eliminar la necesidad de avisar manualmente cada cambio, mediante una plataforma única que registre la agenda, la asignación y el estado de cada servicio.

2. Objetivos y alcance

Objetivo general

Desarrollar y validar un MVP de gestión operativa que permita centralizar la agenda de servicios, asignar trabajos a técnicos y registrar su estado desde dispositivos móviles, con la meta de reducir en al menos 50 % el tiempo dedicado a planificación y coordinación manual durante una prueba piloto simulada.

Objetivos específicos y métricas

Objetivo	Métrica definida
OE1 Centralizar la agenda	100 % de los servicios del piloto registrados en el sistema.
OE2 Digitalizar la asignación	Al menos 95 % de los servicios asignados desde el sistema; menos de 1 minuto por servicio.
OE3 Autonomía del técnico	Al menos 90 % de los servicios con estado actualizado; menos de 30 segundos por actualización.
OE4 Reducir la coordinación manual	Al menos 50 % menos de mensajes y llamadas fuera del sistema.
OE5 Reducir el tiempo administrativo	50 % menos de tiempo, comparando proceso manual y sistema con el mismo escenario.
OE6 Viabilidad técnica	Al menos 95 % de operaciones críticas exitosas; sin pérdida de información ante errores de conexión.

Alcance finalmente implementado

- **Administrador:** inicio de sesión; alta de clientes y ubicaciones (con coordenadas); alta de servicios; planes de mantenimiento (semanal, quincenal, mensual) con pausa y reactivación; generación de servicios a partir de los planes; asignación, reasignación, reprogramación y cancelación; agenda con filtro por día; ruta sugerida por técnico y día; consulta de la ubicación registrada por el técnico; notificaciones.
- **Técnico (desde el celular):** inicio de sesión; servicios del día; datos del cliente, llamada y enlace al mapa; estados Pendiente → En camino → En servicio → Finalizado, y No realizado con motivo; observaciones; notificaciones de los cambios de agenda.
- **Sistema:** historial de cada cambio; bloqueo optimista; control de superposición de agenda; registro mínimo de ubicación del celular (solo al pasar a En servicio y a Finalizado); actualización por consulta periódica.

Funcionalidades fuera del proyecto y razones

Fuera del alcance	Razón
Portal o avisos directos al cliente	El cliente es un stakeholder, no un usuario; reduce el alcance del MVP.
Avisos en tiempo real (WebSockets, push, SMS, correo)	Se evitó infraestructura adicional; se usa consulta periódica cada 30 segundos y al volver a la pestaña.
Ruteo con servicio externo de mapas y ventanas horarias	Se resolvió una versión simple, con distancia en línea recta (Haversine), sin dependencias externas.
Seguimiento continuo por GPS	Decisión de privacidad y de alcance: solo se registra la posición en dos momentos.
Reasignación automática, IA predictiva, aplicación nativa,	No están ligados a una hipótesis ni a una métrica del MVP.

facturación, pagos, chat, integración compleja con WhatsApp	
Generación programada de servicios	La generación se dispara a mano desde la pantalla de planes; queda como trabajo futuro.
Validación con una empresa real	El equipo no tuvo acceso a una empresa; el piloto es simulado.

3. Análisis y modelo

Entidades y relaciones

Un **cliente** tiene una o más **ubicaciones** (piscinas) con dirección y coordenadas. Una ubicación puede tener un **plan de mantenimiento** que genera **servicios** (cada visita concreta) según su frecuencia; también puede recibir servicios puntuales sin plan. Cada servicio puede tener un técnico (un **usuario** con rol técnico) y deja un rastro en el **historial de estados**. Las **notificaciones** son avisos internos dirigidos a un usuario y vinculados al servicio que los originó.

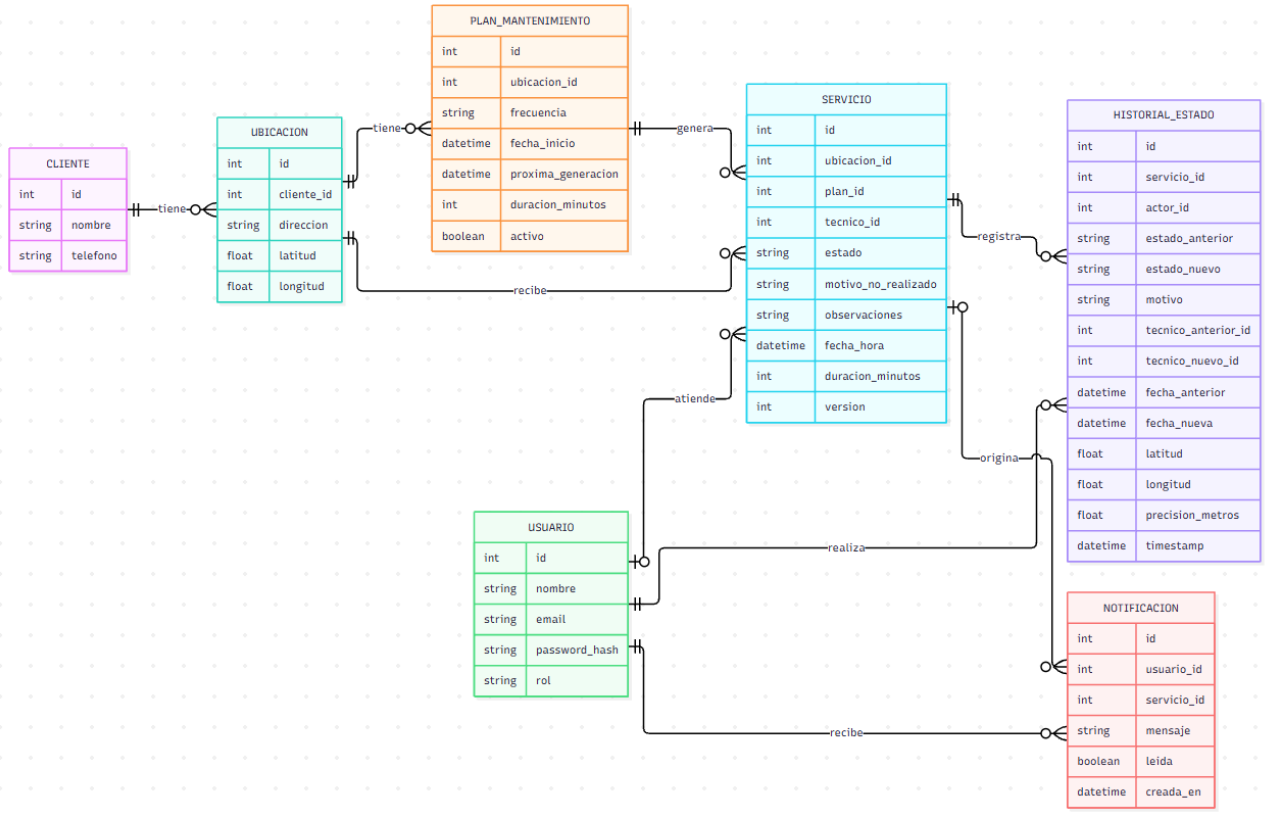


Figura 1. Modelo entidad-relación (12 migraciones incrementales en `db/migrations/`).

Reglas de negocio relevantes

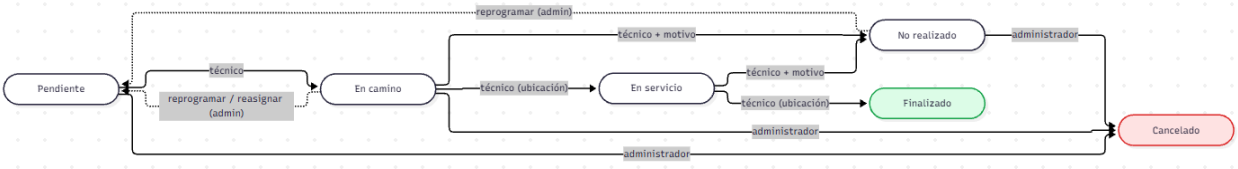


Figura 2. Estados del servicio y quién puede cada transición. Las líneas punteadas son acciones administrativas que devuelven el servicio a Pendiente.

- **Transiciones válidas.** El técnico solo avanza en orden (no se puede saltar de En camino a Finalizado). No realizado exige motivo y está disponible desde En camino o En servicio. El administrador cancela y reprograma. Todo lo demás se rechaza.

- **Reprogramado y reasignado no son estados**, son acciones del administrador que devuelven el servicio a Pendiente y quedan en el historial con fecha anterior y nueva, o técnico anterior y nuevo. Evita mantener dos modelos distintos entre documentación, base y código.
- **Concurrencia.** Toda operación que modifica un servicio usa bloqueo optimista con el campo `version`: el UPDATE exige la versión leída y, si no coincide, se responde 409 Conflict. El cambio y su registro en el historial ocurren en la misma transacción.
- **Superposición de agenda.** Al asignar o reprogramar, el backend verifica que el servicio no se superponga con otro servicio activo del mismo técnico, considerando la duración. Si hay superposición, responde 409.
- **Recurrencia idempotente.** Los servicios se generan a partir de la fecha de inicio y la próxima generación del plan. La restricción única `(plan_id, fecha_hora)` hace que generar dos veces no cree duplicados. Pausar un plan detiene la generación futura sin borrar servicios ya creados.
- **Ubicación.** Se registra solo al pasar a En servicio y a Finalizado. En cualquier otro cambio el backend descarta la ubicación aunque el cliente la envíe, y si el técnico no da permiso el cambio de estado se hace igual.
- **Notificaciones atómicas.** Se crean en la misma transacción que el cambio que las provoca: no existe un aviso de algo que no ocurrió ni un cambio sin su aviso.

Decisiones de modelado especialmente importantes

- **El servicio referencia la ubicación, no al cliente.** Un cliente puede tener varias piscinas y la ruta se calcula hacia el lugar físico.
- **Separar plan y servicio.** Asumiendo que la mayoría de las visitas es recurrente, se modela el plan (la regla) y el servicio (cada visita). Un servicio generado puede modificarse o cancelarse sin afectar al resto del plan.
- **Historial como registro de eventos.** `historial_estado` guarda cambios de estado, reprogramaciones y reasignaciones, y las coordenadas de llegada y finalización, sin sobrescribir nada.
- **Dos tratamientos del tiempo.** Los horarios de agenda son hora local de la empresa, sin conversión de zona horaria (lo que carga el administrador es lo que ve el técnico). Los instantes de los eventos del historial se guardan como momento absoluto (`TIMESTAMPTZ`).

4. Arquitectura y tecnologías

Arquitectura final

Aplicación monolítica de tres capas, sin microservicios: una aplicación de página única en el navegador, una API REST y una base relacional.

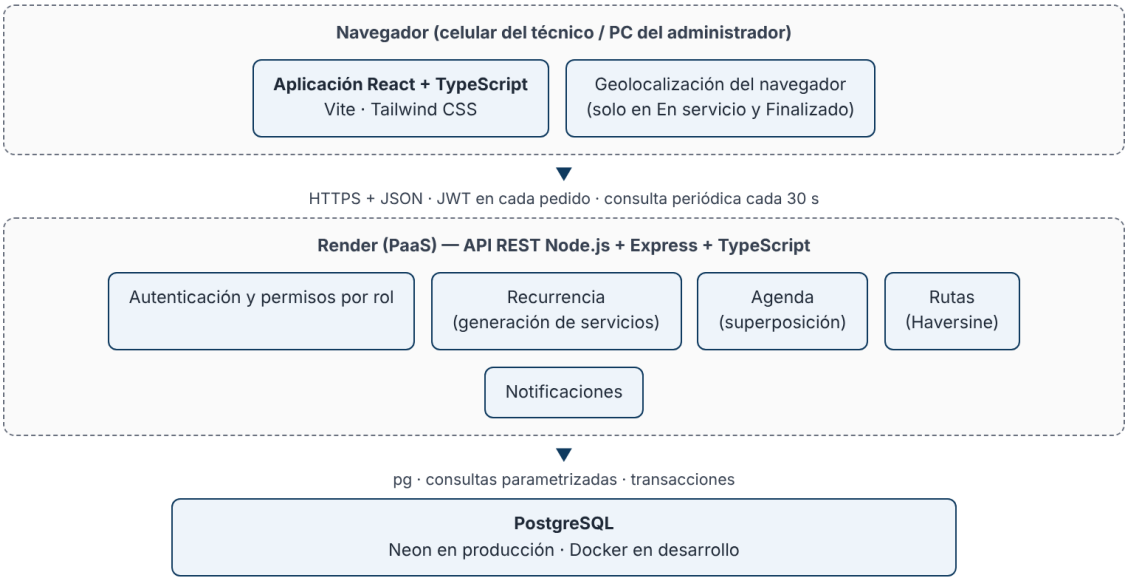


Figura 3. Arquitectura final.

Tecnologías, motivo y responsabilidad

Componente	Responsabilidad	Motivo de la elección
React + TypeScript (Vite)	Interfaz responsiva: pantallas del administrador y del técnico.	Un solo lenguaje en las dos capas y tipado estático para detectar errores. Se usa en el celular del técnico y en la PC del dueño.
Tailwind CSS	Estilos.	Se incorporó al final para cumplir con la interfaz responsiva sin mantener hojas de estilo propias.
Node.js + Express + TypeScript	API REST, reglas de negocio, autorización.	Ecosistema maduro para APIs HTTP y conocimiento previo de JavaScript. No se elige por alta concurrencia, que no es un requisito de una empresa de este tamaño.
PostgreSQL	Persistencia, integridad y transacciones.	Los datos tienen estructura clara y relaciones importantes; se necesitan transacciones y restricciones (únicas, de rango).
JWT + bcrypt	Sesión y almacenamiento de contraseñas.	Autenticación sin estado, sencilla para un MVP; contraseñas guardadas con hash.
Jest + Supertest	Pruebas automatizadas contra la API.	Combinación estándar de Node/Express; permite probar endpoints sin levantar un servidor real.
Render + Neon	Despliegue del frontend y del backend, y base gestionada.	Una PaaS reduce la complejidad operativa. Docker se usa solo en desarrollo local para levantar PostgreSQL.

Integraciones

La única integración es la geolocalización del navegador (`navigator.geolocation`), que solo funciona sobre HTTPS o localhost. El enlace al mapa se abre fuera de la aplicación. No se usa ningún servicio externo de mapas, ruteo, correo ni mensajería.

Seguridad, autorización, concurrencia y sincronización

- **Autenticación y autorización:** el inicio de sesión devuelve un JWT con vencimiento de 8 horas; todos los endpoints lo exigen. Las operaciones de escritura y las consultas que exponen datos de clientes están restringidas al rol administrador. El técnico solo puede ver (`/mis-servicios`) y modificar los servicios que tiene asignados, aunque use la API directamente.
- **Datos:** consultas parametrizadas, validación de entradas (rangos de coordenadas, fechas, estados), restricciones en la base de datos y contraseñas con hash.
- **Concurrencia:** bloqueo optimista por versión y transacciones (ver sección 3).
- **Sincronización:** por consulta periódica, no en tiempo real. El administrador ve cambios al volver a la pestaña, cuando llega una notificación y cada 30 segundos.
- **Decisiones conscientes con límite conocido:** JWT guardado en `localStorage` (en producción se evaluaría una cookie `httpOnly`), CORS abierto (en producción se restringiría a los orígenes del frontend) y backend en un plan gratuito de Render que se duerme tras un período sin uso (la primera respuesta puede tardar hasta un minuto).

5. Implementación

Principales módulos

Módulo	Responsabilidad
`backend/src/index.ts`	Endpoints y reglas: clientes, ubicaciones, servicios (alta, asignación, estado, cancelación, reprogramación, observaciones), planes, ruta sugerida y notificaciones.
`middleware/auth.ts`	Verificación del JWT y del rol.
`recurrencia.ts`	Generación idempotente de servicios a partir de los planes.
`agenda.ts`	Detección de superposición de horarios por técnico.
`rutas.ts`	Distancia de Haversine y orden de visita por vecino más cercano.
`notificaciones.ts`	Creación de avisos dentro de la transacción del cambio.
`frontend/src/*Panel.tsx`	Pantallas del administrador (alta de clientes y servicios, lista de servicios, planes, ruta sugerida) y del técnico.

`ubicacion.ts`, `api.ts`, `fechas.ts`	Obtención de la ubicación del celular, cliente HTTP y formato de fechas (DD/MM/AAAA y HH:MM).
`backend/src/scripts/`	Migraciones, creación de usuarios y herramientas del piloto (carga, limpieza, medición).

Recorridos importantes

Recorrido principal: el administrador crea cliente y ubicación → crea un plan o un servicio → el sistema genera los servicios → el administrador asigna técnico → el técnico inicia sesión y consulta sus servicios → cambia el estado hasta finalizar → el administrador visualiza el resultado. Si el administrador reprograma, reasigna o cancela, el técnico recibe el aviso dentro de la aplicación; si el técnico marca No realizado con motivo, los administradores reciben el aviso.

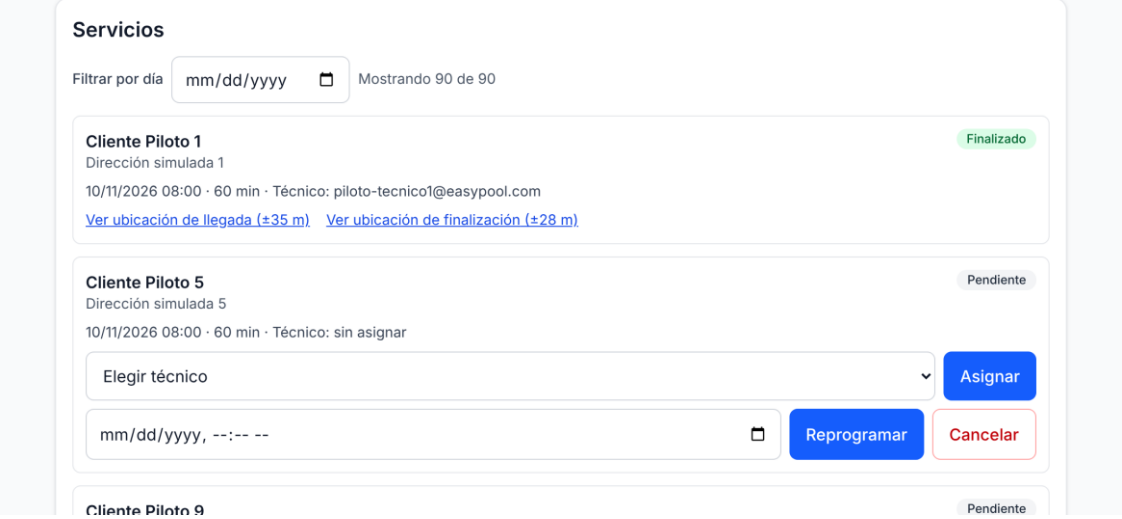


Figura 4. Lista de servicios del administrador, con el enlace a la ubicación registrada al llegar y al finalizar.

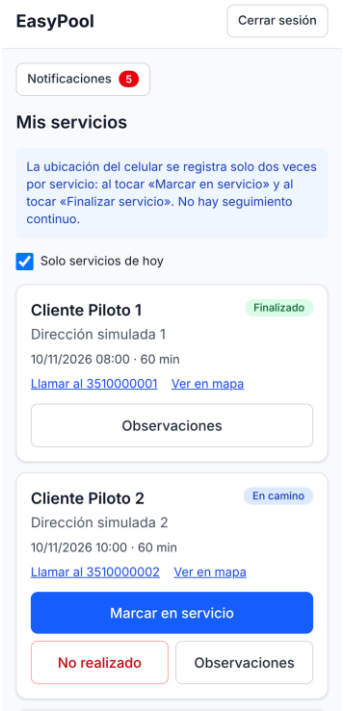


Figura 5. Pantalla del técnico en el celular.



Figura 6. Avisos de asignación y reprogramación.



Figura 7. Ruta sugerida: orden actual contra orden sugerido, con la aclaración de que es una propuesta en línea recta.

Las capturas de las figuras 4 a 7 se tomaron en un entorno local con datos de prueba simulados (clientes «Cliente Piloto N»).

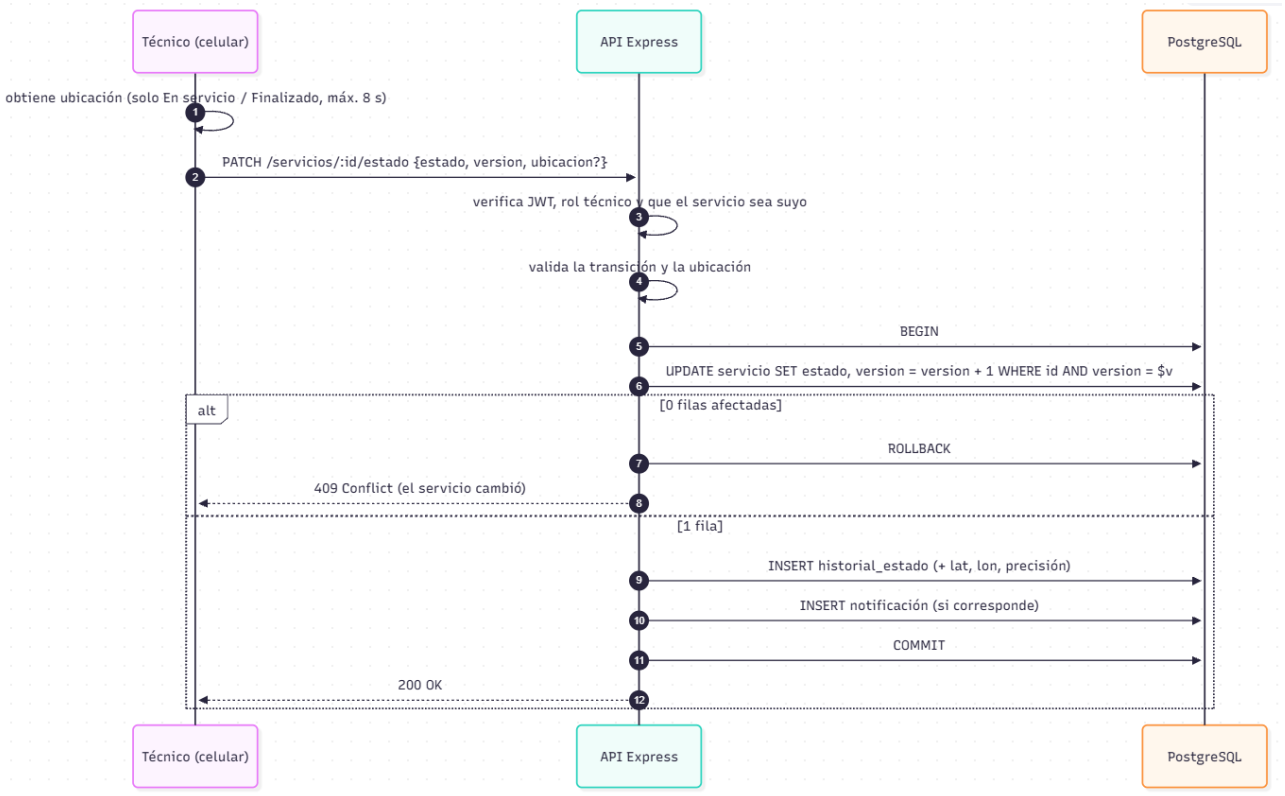


Figura 8. Secuencia de un cambio de estado: validación, bloqueo optimista, historial y notificación en una sola transacción.

Ruta sugerida

El sistema propone un orden de visita para los servicios activos que un técnico tiene en un día. El primer servicio por horario queda fijo como punto de partida, ya que no hay una base de la empresa cargada, y desde ahí se visita siempre el servicio pendiente más cercano (heurística del vecino más cercano), con distancia de Haversine en línea recta entre las coordenadas cargadas por el administrador. No reasigna ni cambia horarios: muestra el orden actual y el sugerido con la diferencia en kilómetros, y el administrador decide. La distancia en línea recta no es la distancia por calles y la heurística no garantiza el recorrido óptimo; en algunos casos el orden sugerido puede igualar o incluso superar al actual, y la pantalla lo informa.

Dificultades técnicas relevantes y cómo se resolvieron

Dificultad	Resolución
Curva de aprendizaje: el equipo no tenía experiencia previa con React ni con Node/Express.	Spike técnico al inicio (inicio de sesión → endpoint protegido → frontend → persistencia) antes de construir funcionalidad, con criterio de reemplazo del stack definido de antemano. El spike resultó exitoso y se mantuvo el stack.
Conflictos entre administrador y técnico sobre el mismo servicio.	Bloqueo optimista con `version` y transacción única para el cambio y su historial.
Servicios recurrentes duplicados o superpuestos.	Generación idempotente con restricción única `(plan_id, fecha_hora)`, control de superposición y pruebas específicas.
Fechas con corrimientos de zona horaria.	Horarios de agenda como hora local sin conversión e instantes del historial como `TIMESTAMPTZ`, con prueba que verifica la hora devuelta.
La ubicación del celular no se guardaba al principio.	Se depuró con la consola del navegador y el pedido de red hasta confirmar que el permiso y los datos llegaban al backend; el registro nunca bloquea el cambio de estado.
Hallazgos del piloto: el orden de la lista cambiaba entre actualizaciones cuando había servicios con la misma hora, y no había señal de que se estuvieran generando los servicios.	Orden estable con desempate por identificador; indicador y botón deshabilitado durante la generación; mensajes de éxito o error visibles en las acciones del administrador.
Una reasignación se rechazaba sin que se viera el motivo.	La causa era la regla de superposición (409); se agregaron mensajes

	visibles.
Backend gratuito que se duerme.	Se documenta, y se despierta antes de demostraciones y mediciones.

6. Pruebas y validación

Qué se probó

El backend tiene **78 pruebas automatizadas en 10 suites** (Jest + Supertest), ejecutadas con `pnpm test` contra una base de datos de prueba separada. El resultado, verificado sobre una base nueva con las 12 migraciones aplicadas desde cero, es de 78 aprobadas.

Suite	Pruebas	Qué verifica
auth	3	Inicio de sesión correcto, contraseña incorrecta y usuario inexistente.
permisos	9	Acceso por rol (403 y 401), recursos inexistentes (404), validación de coordenadas.
estados	17	Transiciones válidas e inválidas, versión desactualizada, servicio ajeno, cancelación, No realizado con y sin motivo, reprogramación, superposición, registro y descarte de ubicación.
reasignacion	8	Asignación y reasignación con historial, usuario inexistente o no técnico, versión desactualizada, pérdida de acceso del técnico anterior.
tecnico	8	Visibilidad de servicios propios y ajenos, observaciones, hora sin corrimientos de zona horaria.
recurrencia	7	Alta de planes, frecuencia inválida, generación, idempotencia, pausa.
notificaciones	10	Avisos al asignar, reasignar, reprogramar, cancelar y No realizado; sin avisos ante un conflicto de versión; lectura propia y ajena.
rutas	7	Distancia de Haversine y orden por cercanía.
rutaSugerida	6	Endpoint de ruta sugerida: permisos, validaciones y orden propuesto.
formatoFechas	3	Formato DD/MM/AAAA y HH:MM.

Cómo se comprobó que se cumplen las reglas críticas

Regla crítica	Evidencia
Estados	Prueba que rechaza saltar de En camino a Finalizado; prueba de que la base ya no acepta el estado «reprogramado»; No realizado sin motivo se rechaza.
Concurrencia	Pruebas que rechazan el cambio y la asignación cuando la versión enviada está desactualizada. No hay una prueba con pedidos simultáneos reales: se verifica la condición de rechazo, no la carrera.
Autorización	Un técnico no puede cambiar el estado de un servicio ajeno, ni crear clientes, ni listar todos los servicios; un administrador no puede usar los endpoints del técnico; sin token se responde 401.
Superposición	Pruebas de rechazo (409) cuando se pisan horarios y de aceptación cuando no se pisan.
Recurrencia idempotente	Correr la generación dos veces no crea duplicados; pausar un plan detiene la generación.
Ubicación	Se registra en la llegada; se descarta en otros cambios; se rechaza fuera de rango (400); sin permiso, el cambio de estado funciona igual.
Notificaciones atómicas	Una operación rechazada por conflicto de versión no genera notificaciones.

Piloto simulado: método

El piloto se ejecutó entre el 09/11 y el 10/11/2026 con el sistema desplegado (frontend y backend en Render, base en Neon). El escenario fue de 5 técnicos, 20 clientes y 20 servicios en el día (18 generados por planes semanales y 2 cargados a mano), con un guion de 8 cambios de agenda: 3 reprogramaciones, 2 reasignaciones, 1 cancelación y 2 servicios No realizado informados por el técnico. Se ejecutaron dos corridas del proceso manual (Excel y mensajes) y dos del proceso con el sistema, con el mismo guion.

- **Cronómetro A:** planificación (asignar los 20 servicios y armar o enviar las agendas). **Cronómetro B:** gestión de los 8 cambios, hasta que el último queda registrado en la pantalla del técnico.
- Se cuenta como contacto fuera del sistema cada mensaje o llamada que el proceso requiere fuera de la aplicación.

- La carga inicial de datos (clientes, planes, servicios) queda fuera de ambos cronómetros, porque en los dos procesos es la misma tarea.
- OE1 se comprobó con una consulta a la base; OE3, con un celular real y un cronómetro con una vuelta por cambio (dos tandas de 9 mediciones); OE6, con un script de operaciones contra el backend desplegado y una prueba sin conexión.

Resultados del piloto

Objetivo	Meta	Resultado	Conclusión
OE1	100 % registrados	20 de 20 en ambas corridas (100 %).	Cumple
OE2	≥ 95 % asignados; < 1 min por servicio	20 de 20 asignados; 7,2 s y 7,4 s por servicio.	Cumple
OE3	≥ 90 % actualizados; < 30 s	18 de 18 cambios hechos por el técnico desde el celular; de 3,27 a 12,94 s (mediana 9,22 s en la tanda 1 y 5,63 s en la 2).	Cumple
OE4	≥ 50 % menos contactos	Manual: 23 y 24 contactos fuera del sistema. Sistema: 0 en ambas corridas.	Cumple
OE5	50 % menos de tiempo	23,2 % y 25,6 % menos en total (rango cruzado: 14,0 % a 33,5 %).	Cumple parcialmente
OE6	≥ 95 % exitosas	85 de 85 operaciones (100 %); prueba sin conexión sin pérdida ni duplicado.	Cumple

Tiempo administrativo (OE5)

	Manual	Sistema	Ahorro
Corrida 1, total (A + B)	414,28 s	318,15 s	23,2 %
Corrida 2, total (A + B)	370,12 s	275,54 s	25,6 %
Planificación (A)	156,93 s y 164,69 s	143,71 s y 147,87 s	8,4 % y 10,2 %
Gestión de los 8 cambios (B)	257,35 s y 205,43 s	174,44 s y 127,67 s	32,2 % y 37,9 %

El ahorro se calcula como (manual − sistema) ÷ manual, comparando cada corrida con su par. La meta de 50 % no se alcanzó. La planificación casi no mejora, porque asignar con una regla en una planilla ya es una tarea rápida; la mejora está en la gestión de los cambios de agenda y en la eliminación de los contactos externos. En el proceso manual hubo entre 5 y 6 mensajes de planificación y 18 contactos por los 8 cambios (16 mensajes y 2 llamadas).

Viabilidad técnica (OE6)

El script ejecutó 85 operaciones críticas contra el backend desplegado, todas exitosas, con medianas de entre 374 ms y 1598 ms (medidas desde una PC, con la latencia de red incluida y el servidor gratuito despierto).

Operación	Cantidad	Exitosas	Mediana (ms)	Máximo (ms)
Inicio de sesión	6	6	908	1887
Listar técnicos	1	1	378	378
Listar servicios	1	1	553	553
Asignar servicio	18	18	1598	1647
Listar mis servicios	5	5	374	380
Cambiar a En camino	18	18	1077	1128
Cambiar a En servicio	18	18	1074	1144
Cambiar a Finalizado	18	18	1075	1137

Respecto del tiempo de respuesta, las operaciones de la API tardaron entre 0,4 y 1,6 s y, en el celular, cada cambio de estado completo, con la solicitud de ubicación, tardó entre 3,3 y 12,9 s. Se considera aceptable por estar por debajo de la meta de 30 s definida en OE3; es una valoración del equipo, no una medición de la percepción de usuarios.

Prueba sin conexión. Con el celular en modo avión, al tocar «Marcar en camino» apareció el mensaje «No hay conexión. Probá de nuevo en unos segundos.» y el servicio siguió en Pendiente; al volver la conexión, el cambio se registró y el historial muestra un único registro `pendiente → en_camino`, sin duplicados. La prueba cubre

el caso en que la solicitud no llega al servidor; el caso de respuesta perdida se apoya en el control de versión (409 al reintentar) y no se probó por separado.

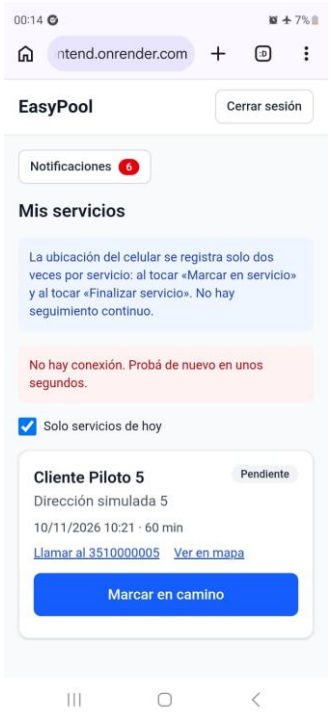


Figura 9. Modo avión: error y servicio en Pendiente.

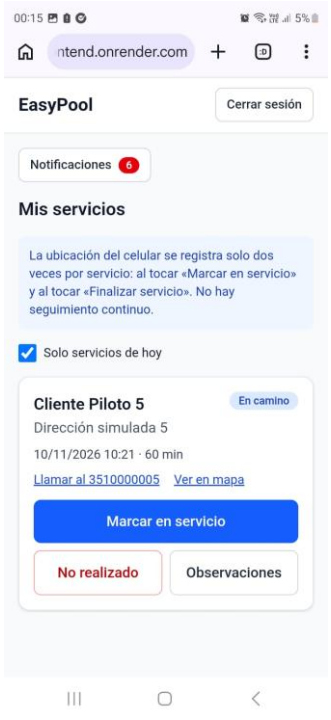


Figura 10. Con conexión: el servicio pasa a En camino.

Los datos de respaldo (planillas, capturas de los cronómetros, salida del script) están en `docs/piloto/` del repositorio.

Limitaciones del piloto

- Lo ejecutó una sola persona, que también hizo de técnicos y de clientes; las respuestas del proceso manual son simuladas y no hay usuarios reales.

- Hay efecto de aprendizaje entre corridas, tanto en el proceso manual como en el sistema.
- Dos corridas por proceso alcanzan para dar un rango, no para estimar variabilidad.
- La carga inicial de datos no entra en ningún cronómetro.
- Los avisos se verificaron después de cada corrida y no se midió cuánto tardan en verse.
- El script de OE6 no envía ubicación en los cambios de estado.
- La ubicación se registra, pero no se validó contra posiciones geográficas reales, ya que los domicilios del piloto son ficticios.
- El escenario asume que el 90 % de los servicios es recurrente: es un supuesto del equipo, no un dato de una empresa real.

7. Resultados y conclusiones

Qué logró resolver el proyecto

El proyecto entrega un sistema funcional que centraliza la agenda, permite asignar y reasignar técnicos, registra el estado de cada servicio desde el celular, genera los servicios recurrentes y avisa los cambios dentro de la aplicación. Esto fue probado con 78 pruebas automatizadas y con un piloto simulado sobre el sistema desplegado.

Qué cambió respecto de la situación inicial

En el escenario simulado, los 8 cambios de agenda requirieron 18 contactos fuera del sistema con el proceso manual y ninguno con el sistema; la gestión de esos cambios fue entre 32 % y 38 % más rápida y el tiempo administrativo total, entre 23 % y 26 % menor. La meta de 50 % de ahorro de tiempo no se alcanzó y el objetivo queda cumplido solo en parte. Estas cifras provienen de una simulación hecha por una persona y no deben leerse como el efecto en una empresa real.

Implementado y probado, y proyectado

Implementado y probado	Proyectado como trabajo futuro
Agenda centralizada, asignación, reasignación, reprogramación y cancelación. Estados del técnico desde el celular, con historial. Planes de mantenimiento y generación idempotente. Bloqueo optimista y control de superposición. Notificaciones dentro de la aplicación (por consulta periódica). Ubicación mínima en dos momentos. Ruta sugerida por distancia en línea recta. Despliegue en línea.	Avisos en tiempo real o por correo. Ruteo con servicio externo de mapas y ventanas horarias. Generación programada de servicios. Cálculo de distancia al domicilio, política de retención de ubicaciones y ubicación en No realizado. Portal o avisos al cliente. Cookie `httpOnly` y CORS restringido. Validación con una empresa real: adopción por técnicos reales y ahorro de tiempo medido.

Limitaciones conocidas

- Sin validación con usuarios ni empresa reales: la adopción y la utilidad percibida no pudieron estimarse.
- Las actualizaciones no son en tiempo real (hasta 30 segundos de demora).
- La ruta sugerida usa distancia en línea recta y una heurística; no es un recorrido óptimo por calles.
- El backend gratuito se duerme tras un período sin uso.
- La prueba de concurrencia verifica el rechazo por versión desactualizada, no una carrera real con pedidos simultáneos.
- El análisis estático del frontend (`lint`) reporta errores de tipado laxo (`any`) que no afectan la compilación.

Aprendizajes y decisiones relevantes

- Definir y validar el stack con un spike antes de construir permitió detectar a tiempo el riesgo de la curva de aprendizaje.

- Poner las reglas críticas (estados, concurrencia, autorización) en el backend y probarlas desde el principio evitó descubrirlas con el piloto.
- El piloto mostró que automatizar la planificación aporta poco por sí sola, y que el valor está en coordinar los cambios; eso orienta el trabajo futuro.
- Declarar cada límite (simulación, una sola persona, estimaciones) forma parte del resultado y no lo debilita.

Distribución del trabajo

Las primeras etapas se desarrollaron en grupo. Tomás José Buforn y Eric Suarez Dubs dejaron el grupo antes del cierre del proyecto; el desarrollo restante, el piloto, la documentación y este informe fueron realizados por Maximiliano Rao.

8. Enlaces de la entrega

Recurso	Enlace
Video de presentación final	https://youtu.be/ZVagtkr7ObY
Repositorio	https://github.com/MaximilianoRao/EasyPool
Versión evaluada (tag o release)	**v1.0-entrega-final**
Sistema desplegado (frontend)	https://easypool-frontend.onrender.com
Sistema desplegado (backend)	https://easypool-backend.onrender.com (el plan gratuito se duerme: el primer pedido puede tardar hasta un minuto)
Cuentas de demostración	admin-demo@easypool.com / 0gk9lXg4F76T tecnico-demo@easypool.com / lSY44PC6uQaU
Documento de análisis y diseño	`Trabajo Integrador Final.md` en el repositorio
Datos del piloto	`docs/piloto/` en el repositorio